

软件设计报告

团队序号： 第22组
项目名称： 自习室座位预约系统
文档版本： V1.0
编写日期： 2026年6月4日

团队成员：

姓名	学号	分工
刘越帅	2313752	系统架构设计、后端开发
朱荟宇	2311824	数据库设计、接口设计
欧一凡	2312826	前端设计、UML建模

目录

- 一、系统总体设计
 - 1.1 项目背景与目标
 - 1.2 系统功能概述
 - 1.3 系统总体架构设计
 - 1.4 技术选型说明
- 二、详细设计
 - 2.1 模块划分与模块功能设计
 - 2.2 核心业务流程设计
 - 2.3 接口设计
- 三、UML设计
 - 3.1 用例图
 - 3.2 类图
 - 3.3 顺序图
 - 3.4 状态图
 - 3.5 部署图
- 四、数据库设计
 - 4.1 数据库E-R图
 - 4.2 主要数据表设计
- 五、其他内容
 - 5.1 安全性设计
 - 5.2 性能优化设计
 - 5.3 可扩展性设计
 - 5.4 异常处理机制
 - 5.5 日志与监控方案

一、系统总体设计

1.1 项目背景与目标

1.1.1 项目背景

随着高校学生数量的增加和自习需求的提升，传统的自习室管理方式（如现场排队、手工登记）已无法满足高效、公平的座位分配需求。学生经常面临以下问题：

- **占座现象严重：**学生提前用物品占座，导致座位资源浪费
- **排队时间过长：**高峰期学生需花费大量时间现场排队
- **座位利用率低：**缺乏实时信息，学生无法知悉座位空闲状态
- **管理效率低下：**管理员手工登记、统计工作量大且易出错

本项目旨在开发一套基于Web的自习室座位预约系统，实现在线预约、实时监控和智能管理，提升座位使用效率和用户体验。

1.1.2 系统目标

1. **便捷预约：**提供在线自习室座位查询与预约服务，学生可随时随地完成预约
2. **资源优化：**实现座位资源的合理分配和高效利用，减少闲置浪费
3. **实时监控：**提供实时的座位状态展示，学生可直观了解自习室使用情况
4. **智能管理：**自动处理违约判定、权限管控、数据统计等管理任务
5. **数据驱动：**通过数据统计分析，为自习室运营决策提供依据

1.2 系统功能概述

系统分为**学生端**和**管理端**两大功能模块：

学生端功能

功能模块	功能描述
用户注册/登录	支持账号密码注册登录，JWT令牌认证
自习室浏览	查看自习室列表、位置、开放时间等信息
座位查看	以可视化方式展示座位布局和实时状态
座位预约	选择座位和时段进行在线预约
预约管理	查看个人预约记录，取消未开始的预约
签到签退	到达后签到确认，离开时签退释放座位
个人信息	查看和修改个人资料，查看违约记录

管理端功能

功能模块	功能描述
用户管理	管理学生账号，设置违约处罚
自习室管理	新增、修改、删除自习室信息

功能模块	功能描述
座位管理	管理座位布局、编号、类型等
预约监控	查看所有预约记录，处理异常预约
数据统计	使用率、活跃度等多维度统计报表
系统设置	配置预约规则、开放时间等参数

1.3 系统总体架构设计

1.3.1 架构风格说明

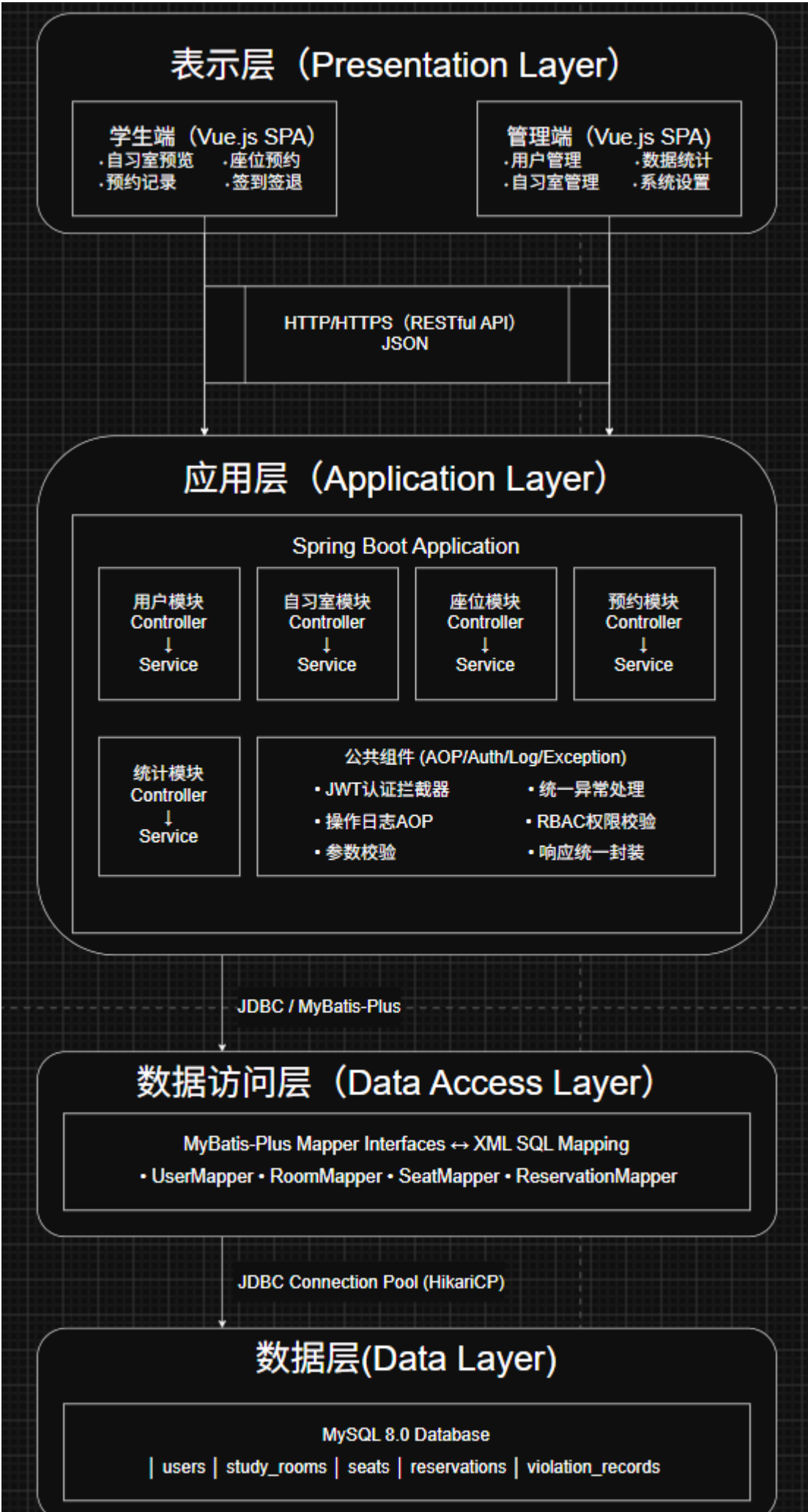
本系统采用**B/S（Browser/Server）** 分层架构，结合**前后端分离**的设计模式。整体分为四层：

- **表示层（Presentation Layer）**：基于Vue.js框架构建的SPA（单页面应用），运行在浏览器中，负责用户界面展示和交互
- **应用层（Application Layer）**：基于Spring Boot构建的RESTful API服务，处理HTTP请求，实现业务逻辑
- **数据访问层（Data Access Layer）**：使用MyBatis-Plus框架实现ORM映射，完成数据库CRUD操作
- **数据层（Data Layer）**：MySQL关系型数据库，存储系统所有业务数据

选择此架构的理由：

1. **分层架构**将系统职责清晰分离，每层只依赖其下层，降低了耦合度
2. **前后端分离**使前端和后端可独立开发、独立部署、独立测试，提高团队协作效率
3. **RESTful API**采用无状态设计，便于水平扩展和负载均衡
4. 各层可独立替换或升级（如更换前端框架或数据库），不影响其他层

1.3.2 系统整体结构图



1.4 技术选型说明

1.4.1 技术栈总览

层次	技术	版本	说明
前端框架	Vue.js	3.4	渐进式JavaScript框架，组件化开发
UI组件库	Element Plus	2.6	成熟的Vue 3组件库，提供丰富UI组件
HTTP客户端	Axios	1.7	Promise-based HTTP客户端
路由	Vue Router	4.3	Vue.js官方路由管理器
状态管理	Pinia	2.1	Vue 3官方推荐状态管理库
后端框架	Spring Boot	3.2	企业级Java开发框架
ORM框架	MyBatis-Plus	3.5	增强版MyBatis，简化CRUD操作
数据库	MySQL	8.0	关系型数据库管理系统
连接池	HikariCP	5.0	Spring Boot默认高性能连接池
认证授权	Spring Security + JWT	-	基于JWT的无状态认证方案
API文档	SpringDoc OpenAPI	2.3	自动生成Swagger API文档
构建工具	Maven	3.9	Java项目依赖管理与构建
前端构建	Vite	5.1	新一代前端构建工具
容器化	Docker	24+	应用容器化部署

1.4.2 选型理由分析

Vue.js 3 + Element Plus（前端）

- Vue 3采用Composition API，代码组织更灵活，逻辑复用性强
- Element Plus提供丰富的企业级UI组件（表格、表单、弹窗、图表等），开发效率高
- Vite构建工具开发体验极佳，热更新速度远超Webpack
- 团队有Vue.js开发经验，学习成本低

Spring Boot 3.2（后端）

- 自动化配置减少了大量样板代码，开发效率高
- 成熟的生态系统（Security、AOP、Validation、Actuator等），满足企业级需求
- 内嵌Tomcat服务器，无需额外部署Web容器
- 课程教学常用框架，符合课程要求

MyBatis-Plus（ORM）

- 在MyBatis基础上提供通用CRUD接口，减少重复编码
- 支持Lambda表达式构建查询条件，类型安全且可读性强

- 与MySQL搭配性能优异，支持复杂SQL自定义

MySQL 8.0（数据库）

- 需求文档明确要求使用MySQL 8.0
- 开源免费，社区活跃，文档丰富
- 支持JSON数据类型和窗口函数等高级特性
- 适合中小型Web应用的数据存储需求

JWT + Spring Security（认证）

- JWT无状态，服务端无需存储会话信息，便于水平扩展
- Spring Security提供完整的认证授权框架，支持RBAC权限模型
- 两者结合可实现安全、高效的用户认证体系

二、详细设计

2.1 模块划分与模块功能设计

2.1.1 模块总体划分

系统共划分为**6个核心模块**，各模块职责明确、边界清晰：



2.1.2 模块职责说明

1. 用户模块 (user-module)

项目	内容
职责	负责用户注册、登录、认证、个人信息管理、违约记录查询
核心功能	用户注册、账号登录、JWT令牌签发与刷新、个人信息修改、密码修改、违约记录查看
输入	注册表单、登录凭证、个人信息编辑表单
输出	JWT令牌、用户信息DTO、违约记录列表

2. 自习室模块 (room-module)

项目	内容
职责	负责自习室信息的增删改查、开放时间设置

项目	内容
核心功能	自习室列表查询、自习室详情查看、自习室新增/修改/删除（管理员）、开放/关闭状态切换
输入	自习室查询条件、自习室编辑表单
输出	自习室列表DTO、自习室详情（含座位概览）

3. 座位模块 (seat-module)

项目	内容
职责	负责座位信息管理、座位状态实时更新
核心功能	座位布局展示、座位状态查询、座位新增/修改/删除（管理员）、座位类型管理、座位二维码生成
输入	自习室ID、座位筛选条件、座位编辑表单
输出	座位列表及状态、座位布局图数据

4. 预约模块 (reservation-module)

项目	内容
职责	负责预约的创建、取消、签到、签退以及违约判定
核心功能	座位预约、取消预约、扫码签到、签退释放、违约自动判定、预约规则校验、时段冲突检测
输入	预约请求（座位ID+时段）、取消请求、签到请求、签退请求
输出	预约记录、签到确认、违约记录、操作结果

5. 统计模块 (statistics-module)

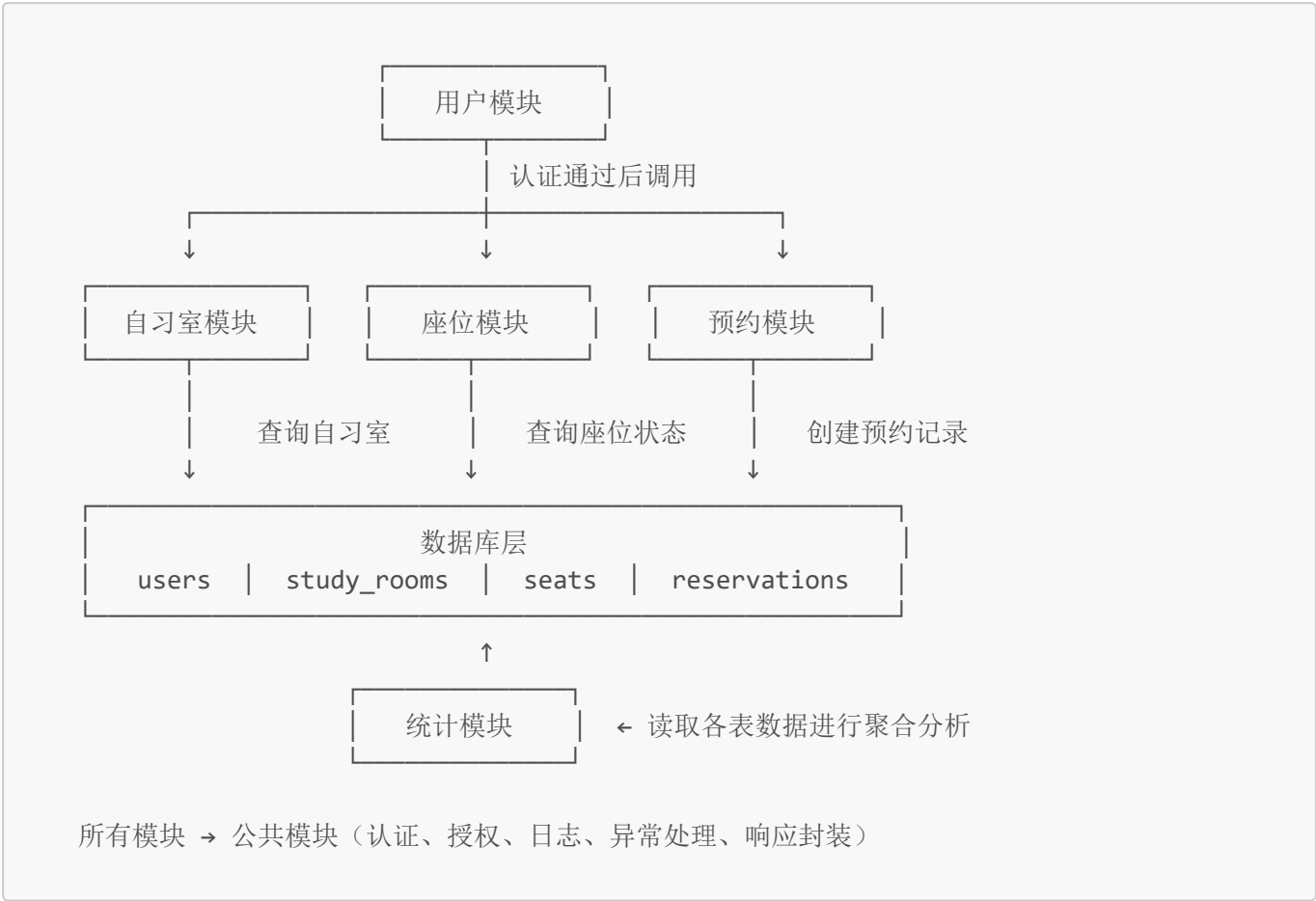
项目	内容
职责	负责系统数据的统计分析和报表生成
核心功能	座位使用率统计、用户活跃度分析、时段热度分析、违约率统计、数据导出
输入	统计时间范围、统计维度选择
输出	统计图表数据、统计报表（Excel导出）

6. 公共模块 (common-module)

项目	内容
职责	提供系统级通用功能和基础设施
核心功能	JWT认证拦截、RBAC权限校验、统一异常处理、统一响应封装、操作日志记录（AOP）、参数校验、跨域配置
输入	HTTP请求、业务异常、操作事件

项目	内容
输出	统一响应格式、日志记录、权限判断结果

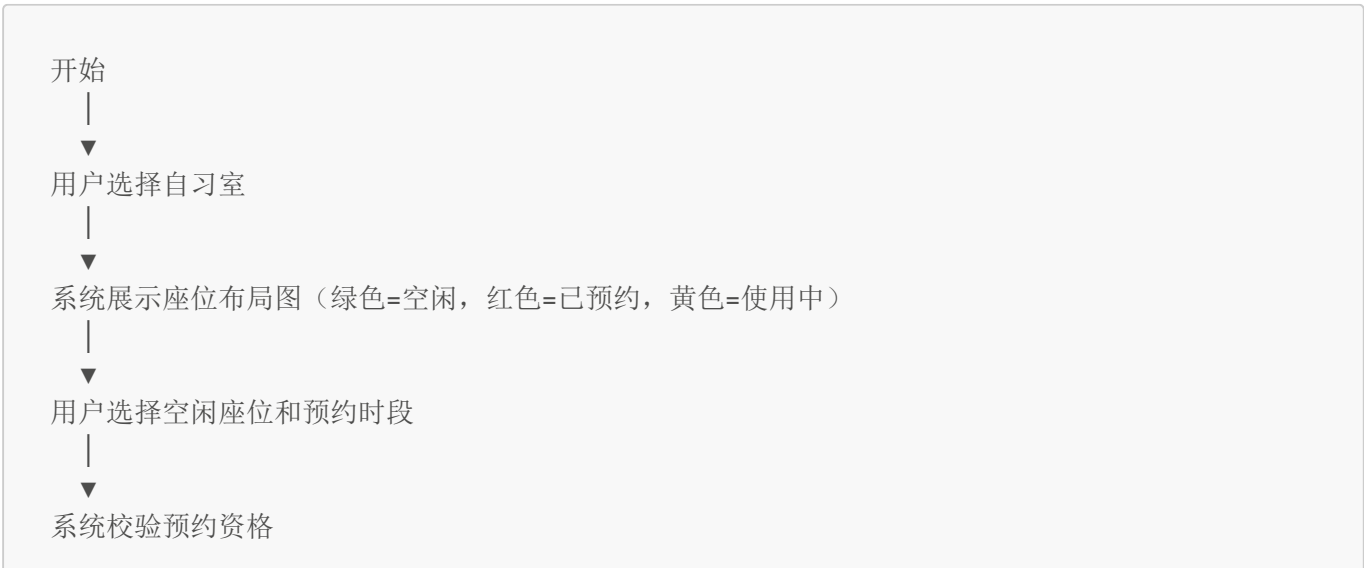
2.1.3 模块调用关系

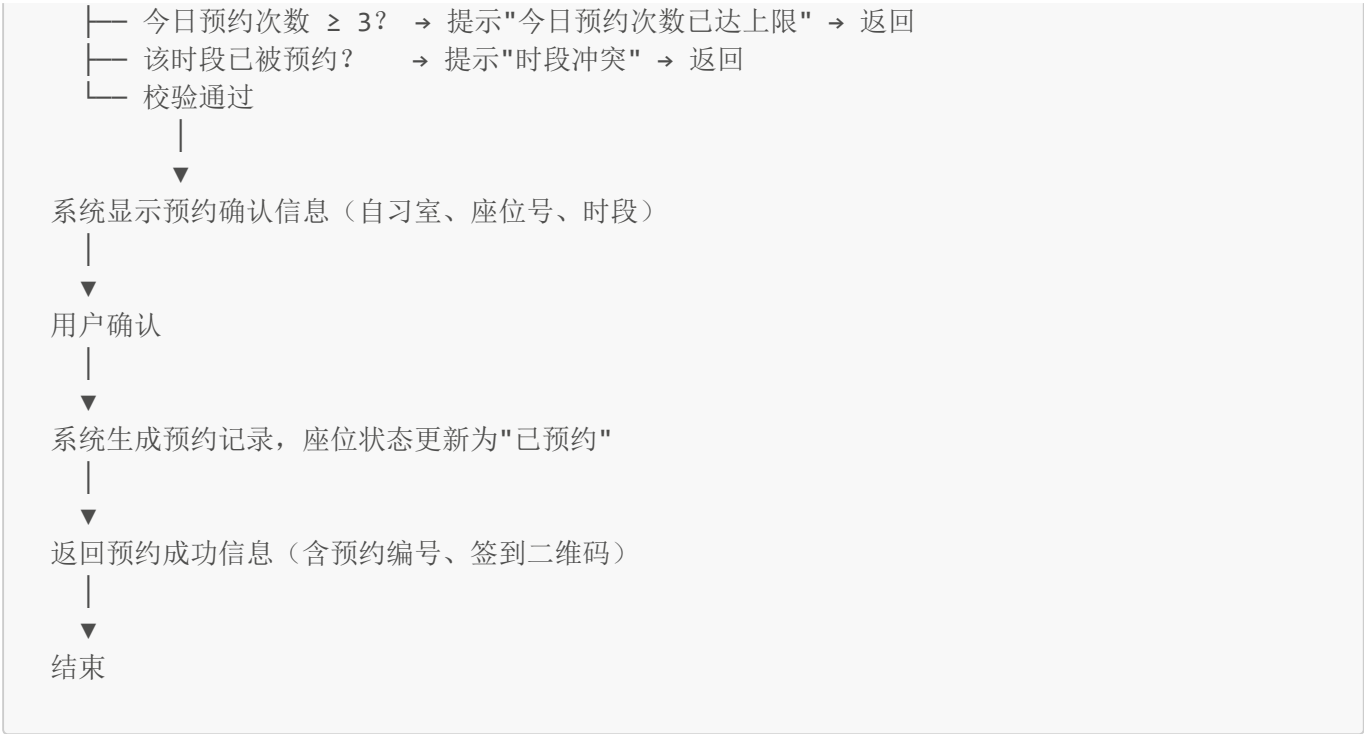


模块间的调用遵循**单向依赖原则**：业务模块（用户、自习室、座位、预约）之间通过Service层进行调用，均依赖公共模块提供的横切功能。统计模块以只读方式访问其他模块的数据，不产生业务写入。

2.2 核心业务流程设计

2.2.1 用户预约座位流程

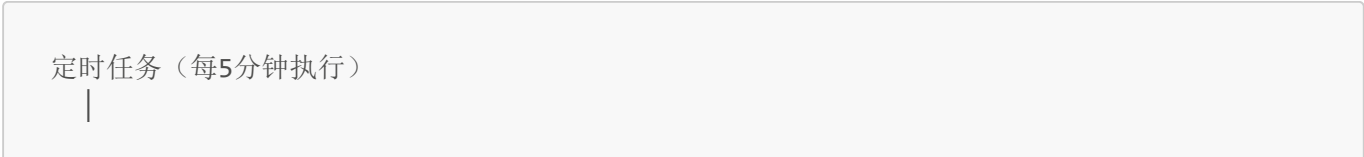




2.2.2 签到流程



2.2.3 违约处理流程





2.2.4 取消预约流程



2.3 接口设计

系统采用RESTful API设计风格，所有接口遵循以下规范：

- 协议：HTTPS
- 数据格式：JSON
- 字符编码：UTF-8
- 认证方式：Header中携带 `Authorization: Bearer <JWT Token>`
- 统一响应格式：

```
{
  "code": 200,
  "message": "success",
  "data": {},
  "timestamp": 1717488000000
}
```

2.3.1 用户模块接口

方法	URL	说明	认证
POST	/api/user/register	用户注册	否
POST	/api/user/login	用户登录，返回JWT	否
GET	/api/user/profile	获取当前用户信息	是
PUT	/api/user/profile	修改个人信息	是
PUT	/api/user/password	修改密码	是
GET	/api/user/violations	查询个人违约记录	是

登录接口示例：

请求：

```
POST /api/user/login
{
  "username": "zhangsan",
  "password": "123456"
}
```

响应：

```
{
  "code": 200,
  "message": "登录成功",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIs...\"",
    "user": {
      "userId": 1,

```

```
    "username": "zhangsan",
    "role": "student",
    "status": 1
  }
}
```

2.3.2 自习室模块接口

方法	URL	说明	权限
GET	/api/rooms	获取自习室列表	学生/管理员
GET	/api/rooms/{id}	获取自习室详情	学生/管理员
POST	/api/rooms	新增自习室	管理员
PUT	/api/rooms/{id}	修改自习室信息	管理员
DELETE	/api/rooms/{id}	删除自习室	管理员
PUT	/api/rooms/{id}/status	切换开放/关闭状态	管理员

2.3.3 座位模块接口

方法	URL	说明	权限
GET	/api/rooms/{roomId}/seats	获取自习室座位布局	学生/管理员
GET	/api/rooms/{roomId}/seats?date=2026-06-04&slot=上午	按日期时段查询座位状态	学生/管理员
POST	/api/rooms/{roomId}/seats	新增座位	管理员
PUT	/api/seats/{id}	修改座位信息	管理员
DELETE	/api/seats/{id}	删除座位	管理员

2.3.4 预约模块接口

方法	URL	说明	权限
POST	/api/reservations	创建预约	学生
GET	/api/reservations/my	查询我的预约记录	学生
DELETE	/api/reservations/{id}	取消预约	学生
POST	/api/reservations/{id}/checkin	签到	学生
POST	/api/reservations/{id}/checkout	签退	学生
GET	/api/reservations/all	查询全部预约（含筛选）	管理员

创建预约请求示例：

```
POST /api/reservations
{
  "seatId": 42,
  "reserveDate": "2026-06-05",
  "timeSlot": "上午",
  "startTime": "2026-06-05 08:00",
  "endTime": "2026-06-05 12:00"
}
```

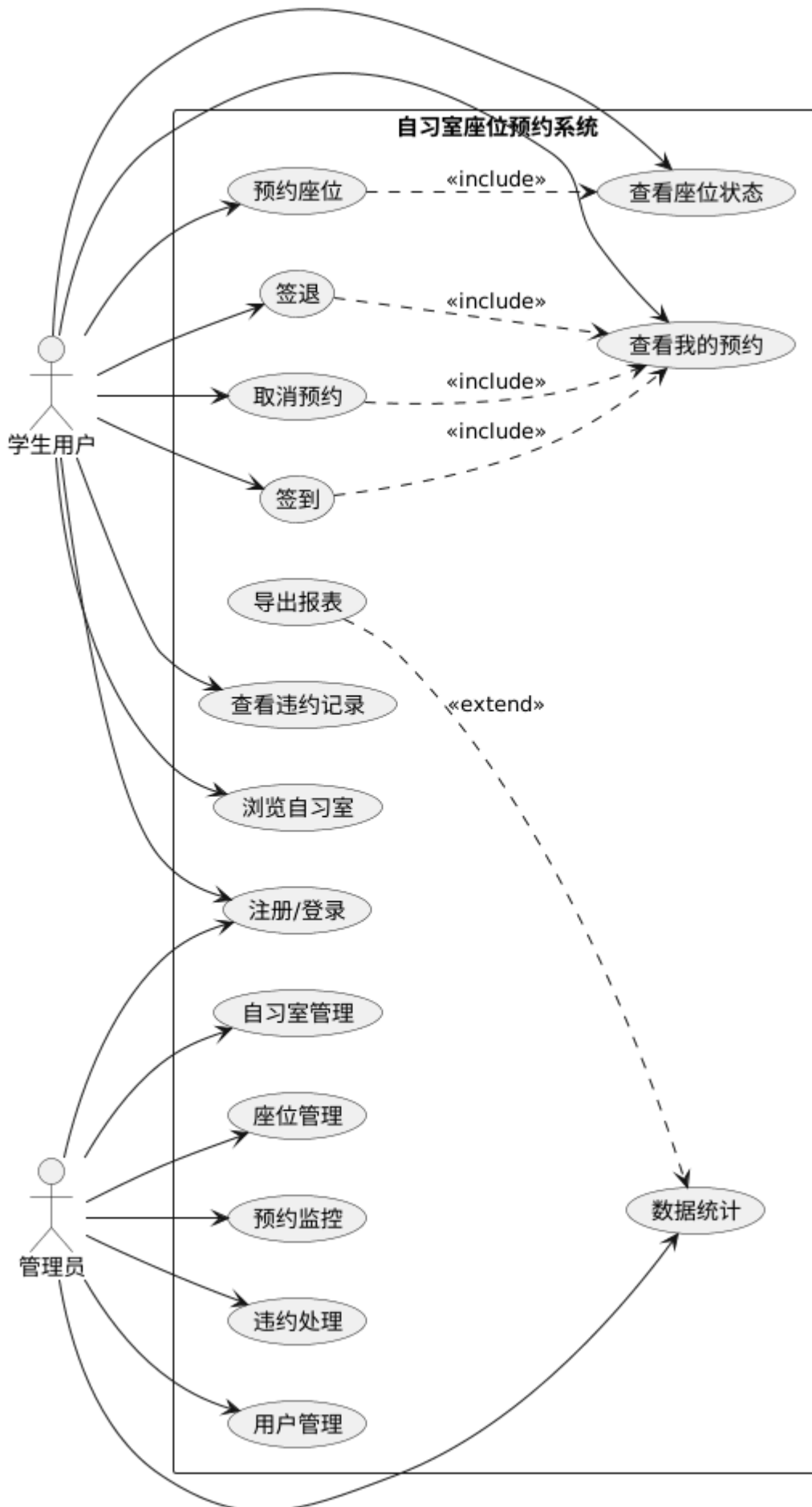
2.3.5 统计模块接口

方法	URL	说明	权限
GET	/api/statistics/usage?start=...&end=...	座位使用率统计	管理员
GET	/api/statistics/active-users?start=...&end=...	用户活跃度统计	管理员
GET	/api/statistics/peak-hours?start=...&end=...	时段热度分析	管理员
GET	/api/statistics/violations?start=...&end=...	违约率统计	管理员
GET	/api/statistics/export?start=...&end=...	导出统计报表	管理员

三、UML设计

3.1 用例图（Use Case Diagram）

本系统涉及两类参与者：**学生用户**和**管理员**。

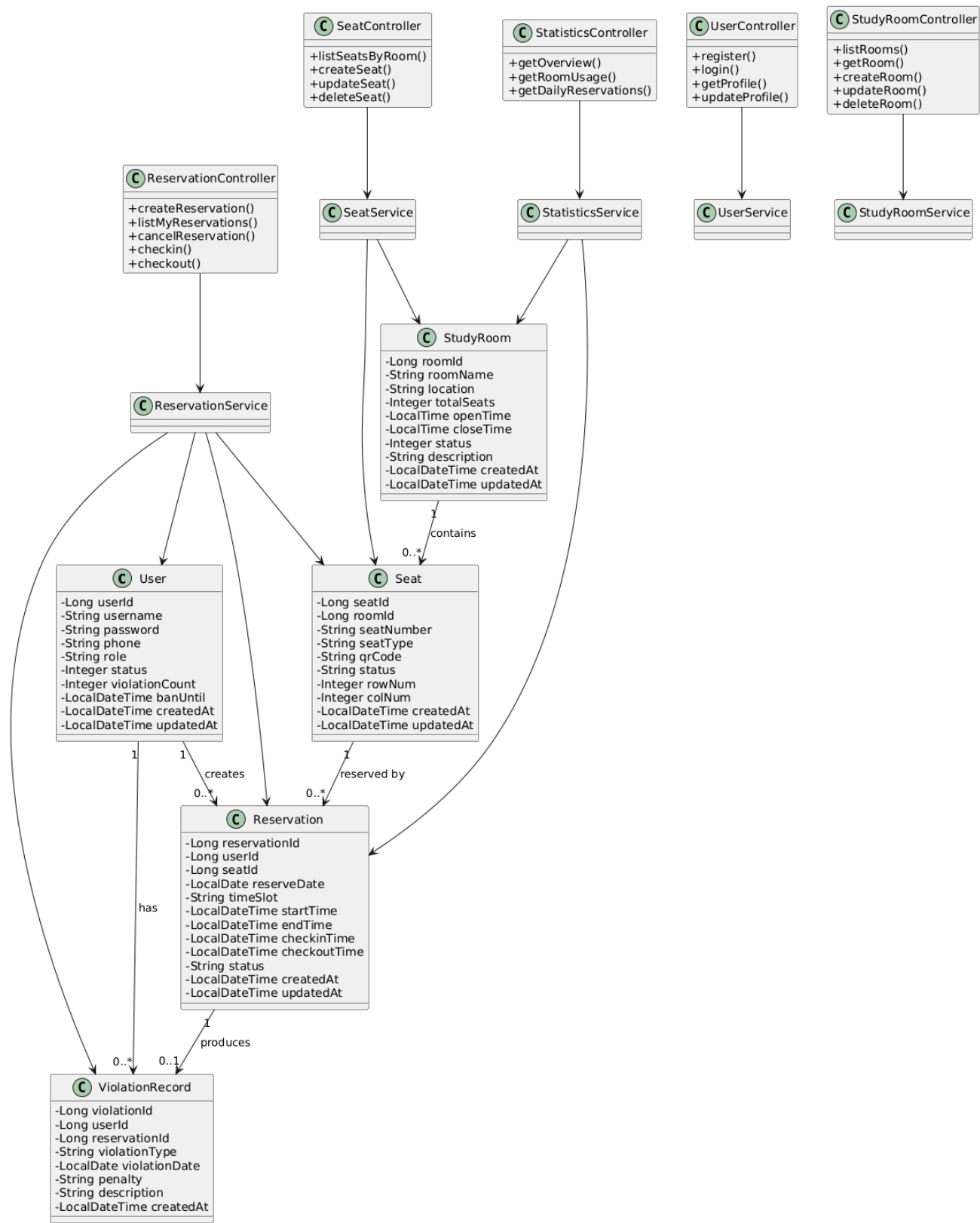


用例详细描述（核心用例）：

用例编号	UC001	用例名称	预约座位
参与者	学生用户		
前置条件	用户已登录，目标座位空闲，今日预约次数未达上限		
后置条件	座位状态变为"已预约"，生成预约记录		
基本事件流	1. 用户选择目标自习室 2. 系统显示座位布局及状态 3. 用户选择空闲座位和预约时段 4. 系统显示预约确认信息 5. 用户确认预约 6. 系统校验通过，生成预约记录 7. 返回预约成功信息		
备选事件流	A1. 时段冲突：系统提示"该时段已被预约"，返回座位选择 A2. 预约超限：系统提示"今日预约次数已达上限" A3. 用户取消确认：返回座位选择页面		

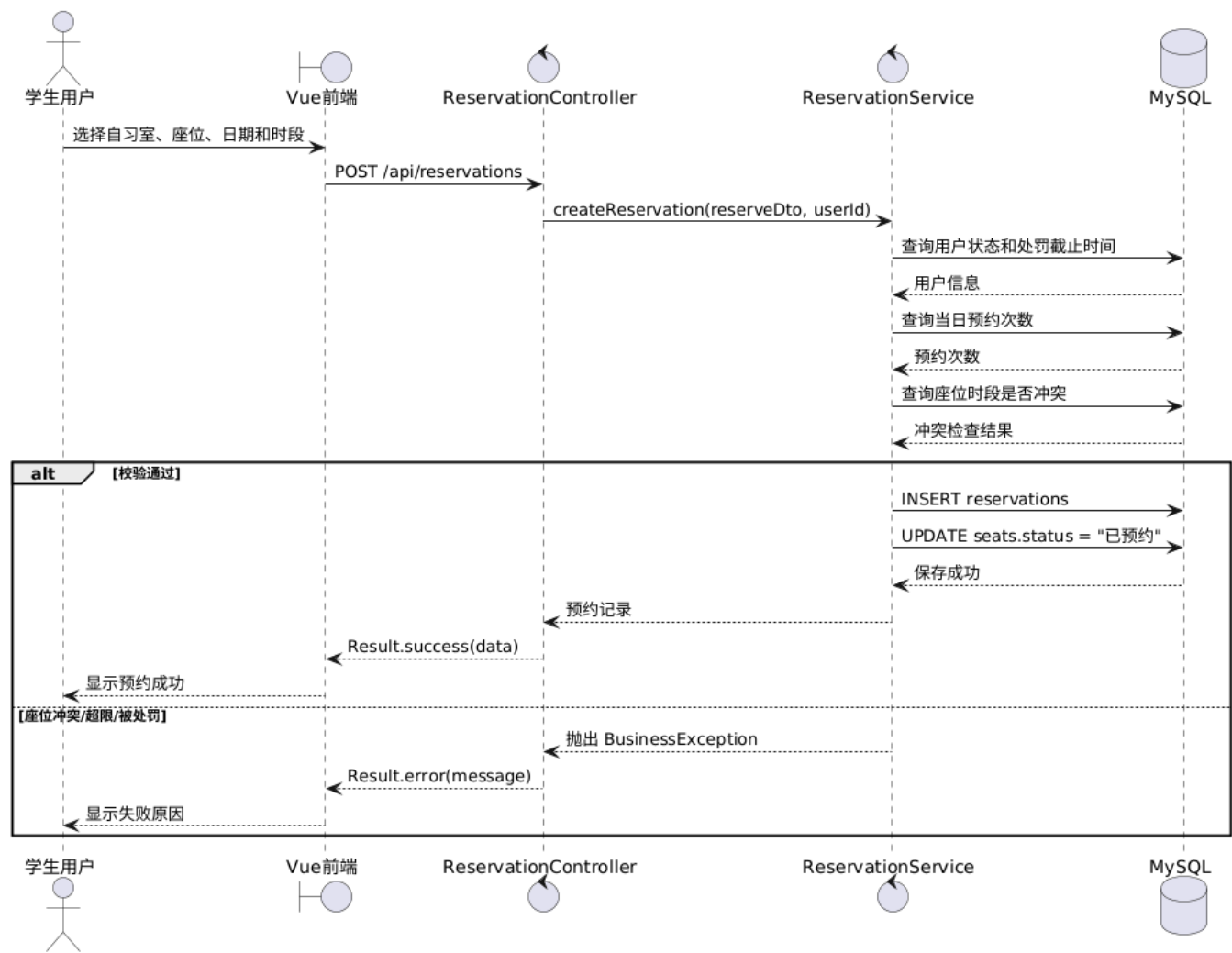
3.2 类图（Class Diagram）

以下展示系统的核心类结构，包括实体类、Service类、Controller类的关系。

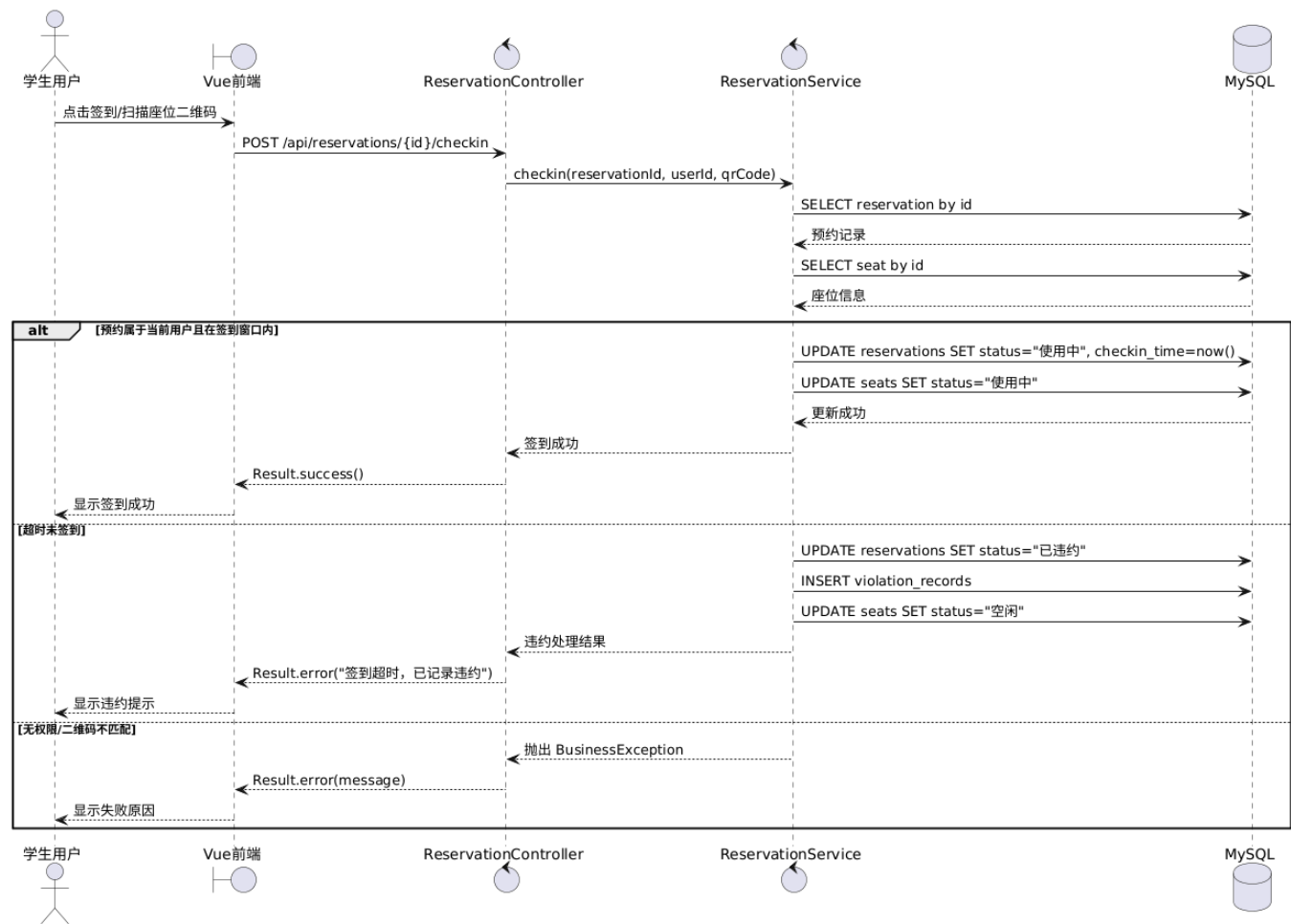


3.3 顺序图 (Sequence Diagram)

3.3.1 预约座位顺序图

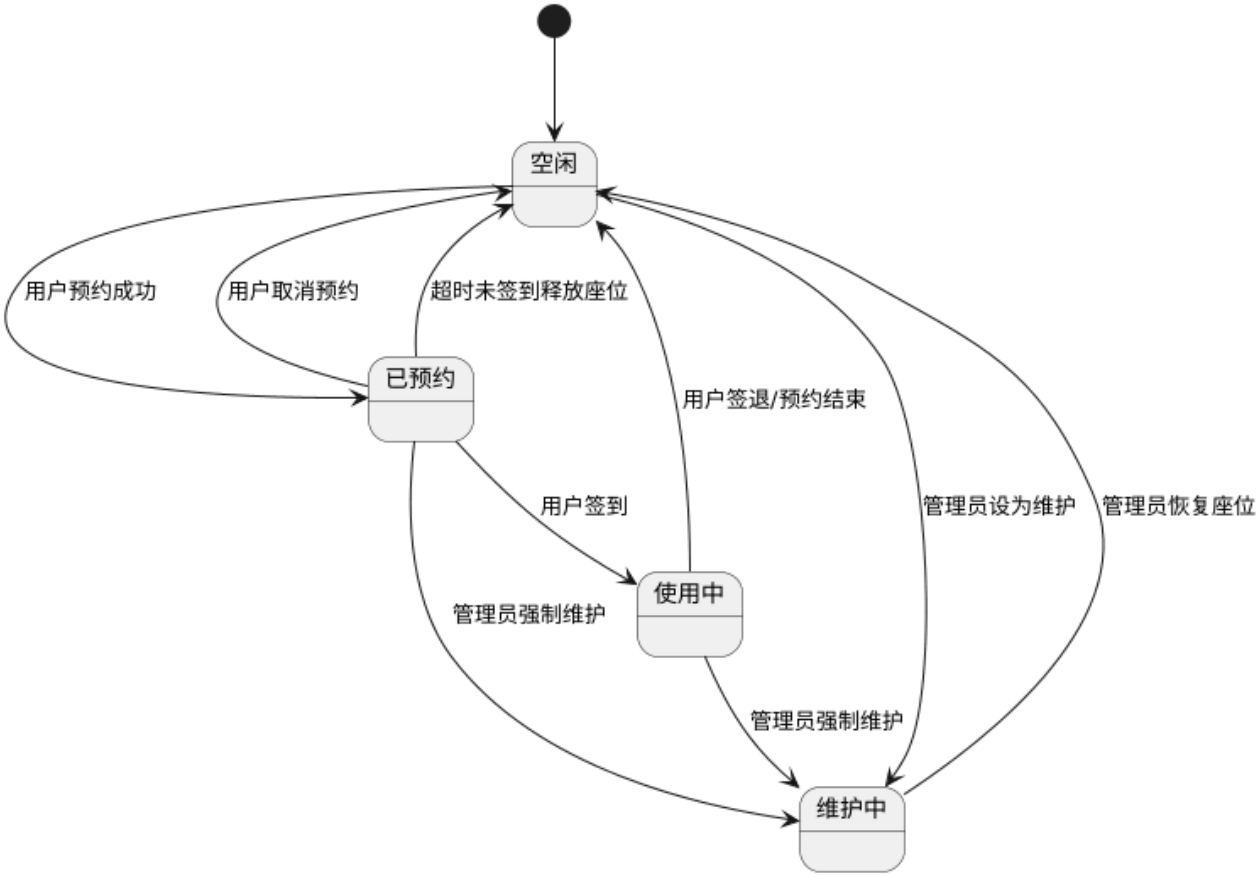


3.3.2 用户签到顺序图

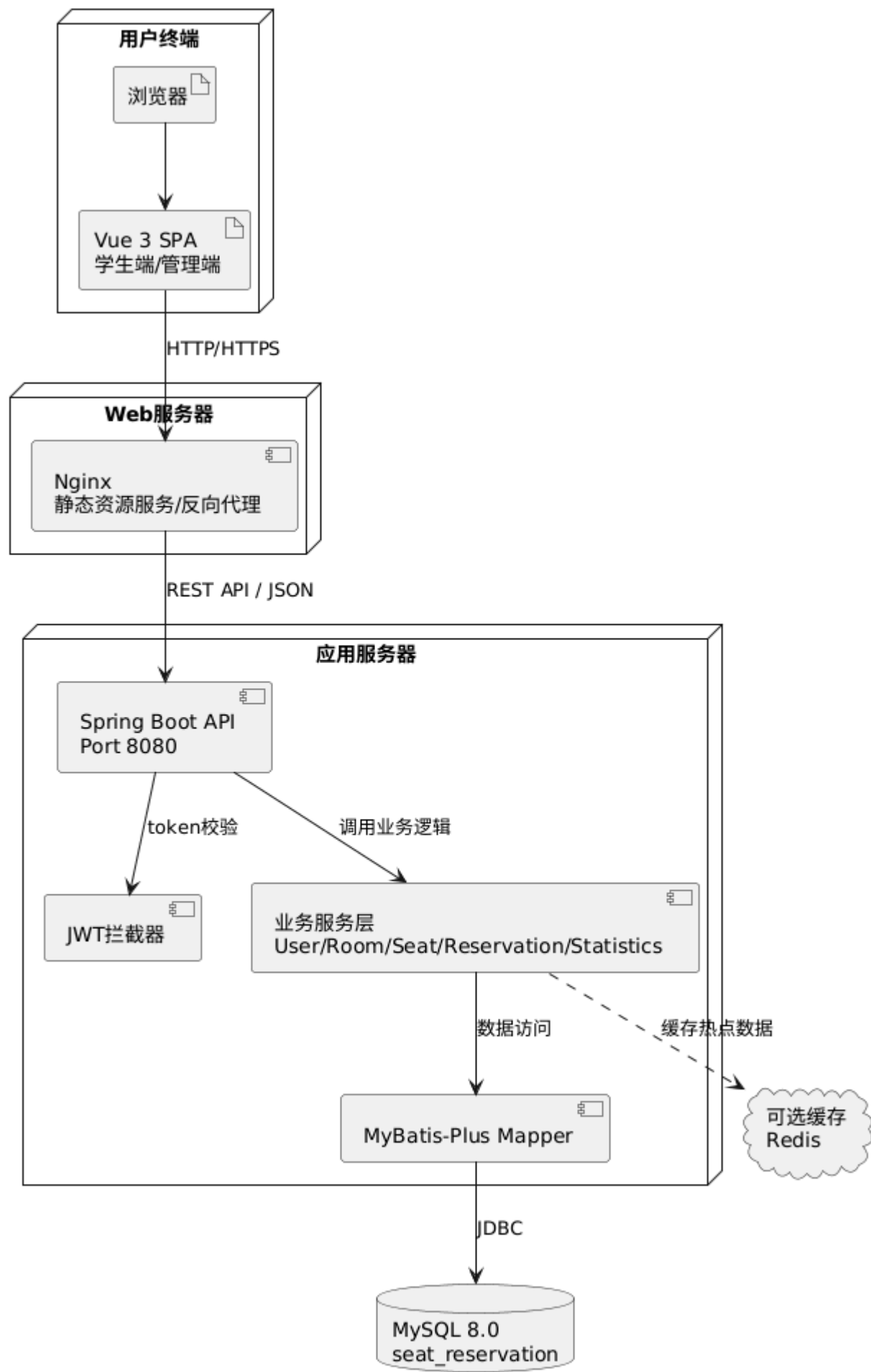


3.4 状态图 (State Diagram)

3.4.1 座位状态转换图



3.5 部署图 (Deployment Diagram)



四、数据库设计

4.1 数据库E-R图



关系说明：

- User ——< Reservation：一个用户可创建多条预约
- Seat ——< Reservation：一个座位可被多次预约（不同时段）
- StudyRoom ——< Seat：一个自习室包含多个座位
- User ——< ViolationRecord：一个用户可有多条违约记录
- Reservation ——< ViolationRecord：一次预约最多对应一条违约记录

4.2 主要数据表设计

4.2.1 用户表 (users)

字段名	数据类型	长度	允许空	约束	说明
user_id	BIGINT	-	否	PK, AUTO_INCREMENT	用户ID
username	VARCHAR	20	否	UNIQUE	用户名
password	VARCHAR	100	否	-	密码 (BCrypt加密)
phone	VARCHAR	15	是	-	手机号码
role	VARCHAR	10	否	DEFAULT 'student'	角色 (student/admin)
status	TINYINT	1	否	DEFAULT 1	状态 (0禁用/1正常)
violation_count	INT	-	否	DEFAULT 0	连续违约次数
ban_until	DATETIME	-	是	-	处罚截止时间
created_at	DATETIME	-	否	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	-	否	ON UPDATE CURRENT_TIMESTAMP	更新时间

4.2.2 自习室表 (study_rooms)

字段名	数据类型	长度	允许空	约束	说明
room_id	BIGINT	-	否	PK, AUTO_INCREMENT	自习室ID
room_name	VARCHAR	50	否	UNIQUE	自习室名称
location	VARCHAR	100	否	-	位置信息 (如: 图书馆2楼)
total_seats	INT	-	否	DEFAULT 0	总座位数
open_time	TIME	-	否	DEFAULT '07:00'	开放时间
close_time	TIME	-	否	DEFAULT '22:00'	关闭时间
status	TINYINT	1	否	DEFAULT 1	状态 (0关闭/1开放)
description	VARCHAR	255	是	-	描述信息
created_at	DATETIME	-	否	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	-	否	ON UPDATE CURRENT_TIMESTAMP	更新时间

4.2.3 座位表 (seats)

字段名	数据类型	长度	允许空	约束	说明
seat_id	BIGINT	-	否	PK, AUTO_INCREMENT	座位ID
room_id	BIGINT	-	否	FK → study_rooms.room_id	所属自习室ID
seat_number	VARCHAR	10	否	-	座位编号（如A01）
seat_type	VARCHAR	20	否	DEFAULT '普通'	座位类型（普通/靠窗/插座）
qr_code	VARCHAR	100	是	UNIQUE	签到二维码标识
status	VARCHAR	20	否	DEFAULT '空闲'	状态（空闲/已预约/使用中/维护中）
row_num	INT	-	是	-	行号（用于布局展示）
col_num	INT	-	是	-	列号（用于布局展示）
created_at	DATETIME	-	否	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	-	否	ON UPDATE CURRENT_TIMESTAMP	更新时间

索引：

- INDEX idx_room_id (room_id)
- UNIQUE INDEX idx_room_seat (room_id, seat_number)

4.2.4 预约记录表（reservations）

字段名	数据类型	长度	允许空	约束	说明
reservation_id	BIGINT	-	否	PK, AUTO_INCREMENT	预约ID
user_id	BIGINT	-	否	FK → users.user_id	用户ID
seat_id	BIGINT	-	否	FK → seats.seat_id	座位ID
reserve_date	DATE	-	否	-	预约日期
time_slot	VARCHAR	10	否	-	时段（上午/下午/晚上）
start_time	DATETIME	-	否	-	开始时间
end_time	DATETIME	-	否	-	结束时间
checkin_time	DATETIME	-	是	-	签到时间
checkout_time	DATETIME	-	是	-	签退时间

字段名	数据类型	长度	允许空	约束	说明
status	VARCHAR	20	否	DEFAULT '已预约'	状态（已预约/使用中/已完成/已取消/已违约）
created_at	DATETIME	-	否	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	-	否	ON UPDATE CURRENT_TIMESTAMP	更新时间

索引：

- INDEX `idx_user_id` (`user_id`)
- INDEX `idx_seat_id` (`seat_id`)
- INDEX `idx_reserve_date` (`reserve_date`)
- INDEX `idx_status` (`status`)
- INDEX `idx_seat_date_slot` (`seat_id`, `reserve_date`, `time_slot`)

4.2.5 违约记录表 (`violation_records`)

字段名	数据类型	长度	允许空	约束	说明
violation_id	BIGINT	-	否	PK, AUTO_INCREMENT	记录ID
user_id	BIGINT	-	否	FK → users.user_id	用户ID
reservation_id	BIGINT	-	否	FK → reservations.reservation_id	关联预约ID
violation_type	VARCHAR	30	否	-	违约类型（迟到未签到/提前离席）
violation_date	DATE	-	否	-	违约日期
penalty	VARCHAR	50	是	-	处罚措施描述
description	VARCHAR	255	是	-	详细说明
created_at	DATETIME	-	否	DEFAULT CURRENT_TIMESTAMP	创建时间

索引：

- INDEX `idx_user_id` (`user_id`)
- INDEX `idx_violation_date` (`violation_date`)

五、其他内容

5.1 安全性设计

5.1.1 身份认证

系统采用JWT（JSON Web Token）+ Spring Security实现用户身份认证：



- JWT令牌包含：userId、username、role、过期时间
- Access Token有效期：2小时
- Refresh Token有效期：7天，用于无感刷新Access Token
- 令牌存储在浏览器localStorage中，每次请求通过Authorization Header携带

5.1.2 权限控制（RBAC）

系统采用基于角色的访问控制（Role-Based Access Control），区分两种角色：

角色	权限范围
student	查看自习室、查看座位、预约座位、取消预约、签到签退、查看个人记录
admin	全部权限：用户管理、自习室管理、座位管理、预约管理、数据统计、系统设置

实现方式：

- 使用Spring Security的@PreAuthorize注解进行方法级权限控制
- 管理员接口使用@PreAuthorize("hasRole('admin')")进行拦截
- 用户只能操作自己的预约记录（数据权限隔离）

5.1.3 数据安全

安全措施	实现方式
密码加密	使用BCrypt算法进行单向加密存储，不可逆
HTTPS传输	生产环境强制HTTPS，配置SSL证书
SQL注入防护	使用MyBatis-Plus参数化查询，杜绝SQL拼接
XSS防护	前端输出编码，后端配置CSP（Content Security Policy）头
CSRF防护	JWT令牌天然防CSRF（不依赖Cookie）
请求频率限制	使用Spring拦截器实现IP级别限流（如登录接口5次/分钟）
会话管理	JWT无状态设计，Token过期自动失效

5.1.4 操作审计

- 使用AOP（面向切面编程）记录所有关键操作日志
- 记录内容：操作人、操作类型、操作时间、IP地址、操作详情
- 日志存储到独立的操作日志表，定期归档

5.2 性能优化设计

5.2.1 数据库优化

优化措施	说明
合理索引	在seat_id、user_id、reserve_date等高频查询字段建立索引
连接池	使用HikariCP连接池，最大连接数20，最小空闲连接数5
分页查询	所有列表查询接口强制分页，默认每页20条，最大100条
慢查询监控	开启MySQL慢查询日志，阈值设为1秒，定期优化

5.2.2 应用层优化

优化措施	说明
缓存策略	自习室列表、座位布局等变化频率低的数据使用Redis缓存，TTL 5分钟
异步处理	统计报表生成、日志记录等非实时操作使用@Async异步执行
定时任务优化	违约判定定时任务每5分钟执行一次，使用分布式锁避免重复执行
前端优化	Vue路由懒加载，组件按需导入，图片压缩，CDN加速静态资源

5.2.3 并发控制

预约场景存在并发竞争（同一座位同一时段可能被多人同时预约），采用以下方案：

- 1. **数据库乐观锁**：在seats表增加version字段，更新时校验版本号
- 2. **唯一索引约束**：reservations表建立(seat_id, reserve_date, time_slot)联合唯一索引
- 3. **业务兜底**：创建预约时在事务中先查询后插入，利用数据库行锁保证一致性

5.3 可扩展性设计

5.3.1 水平扩展

- **无状态服务**：后端API服务不存储会话信息，可水平扩展至多个实例
- **Nginx负载均衡**：使用Nginx配置upstream，分发请求至多个Spring Boot实例
- **数据库读写分离**：预留MyBatis读写分离配置接口，后续可接入MySQL主从复制

5.3.2 模块化设计

- **代码层面**：采用Maven多模块结构（common、service、web），模块间低耦合
- **接口层面**：所有模块通过Service接口通信，实现类可独立替换
- **配置层面**：业务规则（预约次数上限、签到窗口、违约阈值等）抽取为配置项，支持动态调整

5.3.3 预留扩展点

扩展点	预留方式
微信小程序接入	API层与前端解耦，后续可在不修改后端的情况下接入微信小程序
消息通知	预留通知接口（短信/邮件/微信推送），当前先通过系统内消息展示
支付系统	若未来需要付费预约，预留支付接口集成点
第三方登录	认证模块设计预留OAuth2扩展接口

5.4 异常处理机制

5.4.1 异常分类

系统定义三层异常体系：



5.4.2 统一异常处理

使用Spring的@RestControllerAdvice实现全局异常拦截：

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BusinessException.class)
    public Result handleBusinessException(BusinessException e) {
        // 返回业务错误信息，HTTP 409
        return Result.error(409, e.getMessage());
    }

    @ExceptionHandler(ValidationException.class)
    public Result handleValidationException(ValidationException e) {
        // 返回参数校验错误，HTTP 400
        return Result.error(400, "参数校验失败：" + e.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public Result handleSystemException(Exception e) {
```

```
// 记录详细日志，返回统一的系统错误提示，HTTP 500
log.error("系统异常", e);
return Result.error(500, "系统繁忙，请稍后重试");
}
}
```

5.4.3 前端异常处理

- Axios响应拦截器统一处理HTTP错误（401跳转登录、403提示无权限、500通用提示）
- 关键操作使用try-catch捕获异常并展示友好提示
- 网络超时设置10秒，超时后提供重试按钮

5.5 日志与监控方案

5.5.1 日志方案

采用SLF4J + Logback日志框架，按照以下级别和策略记录：

日志级别	记录内容	示例
ERROR	系统异常、数据库连接失败、第三方服务调用失败	堆栈信息、请求参数
WARN	业务警告（违约、权限不足）、降级操作	用户ID、警告类型
INFO	关键业务操作（注册、预约、签到）、定时任务执行	操作人、操作类型、结果
DEBUG	开发调试信息、SQL语句（仅开发环境）	方法入参、返回值

日志输出：

- 控制台输出（开发环境）
- 文件滚动存储（生产环境）：`logs/seat-reservation.%d{yyyy-MM-dd}.log`
- 日志保留30天，单文件最大100MB
- 使用MDC（Mapped Diagnostic Context）记录请求TraceId，便于链路追踪

5.5.2 系统监控

使用Spring Boot Actuator暴露健康检查和指标端点：

端点	说明
<code>/actuator/health</code>	健康检查（数据库连接、磁盘空间）
<code>/actuator/metrics</code>	应用指标（请求数、响应时间、JVM内存）
<code>/actuator/info</code>	应用信息（版本号、构建时间）

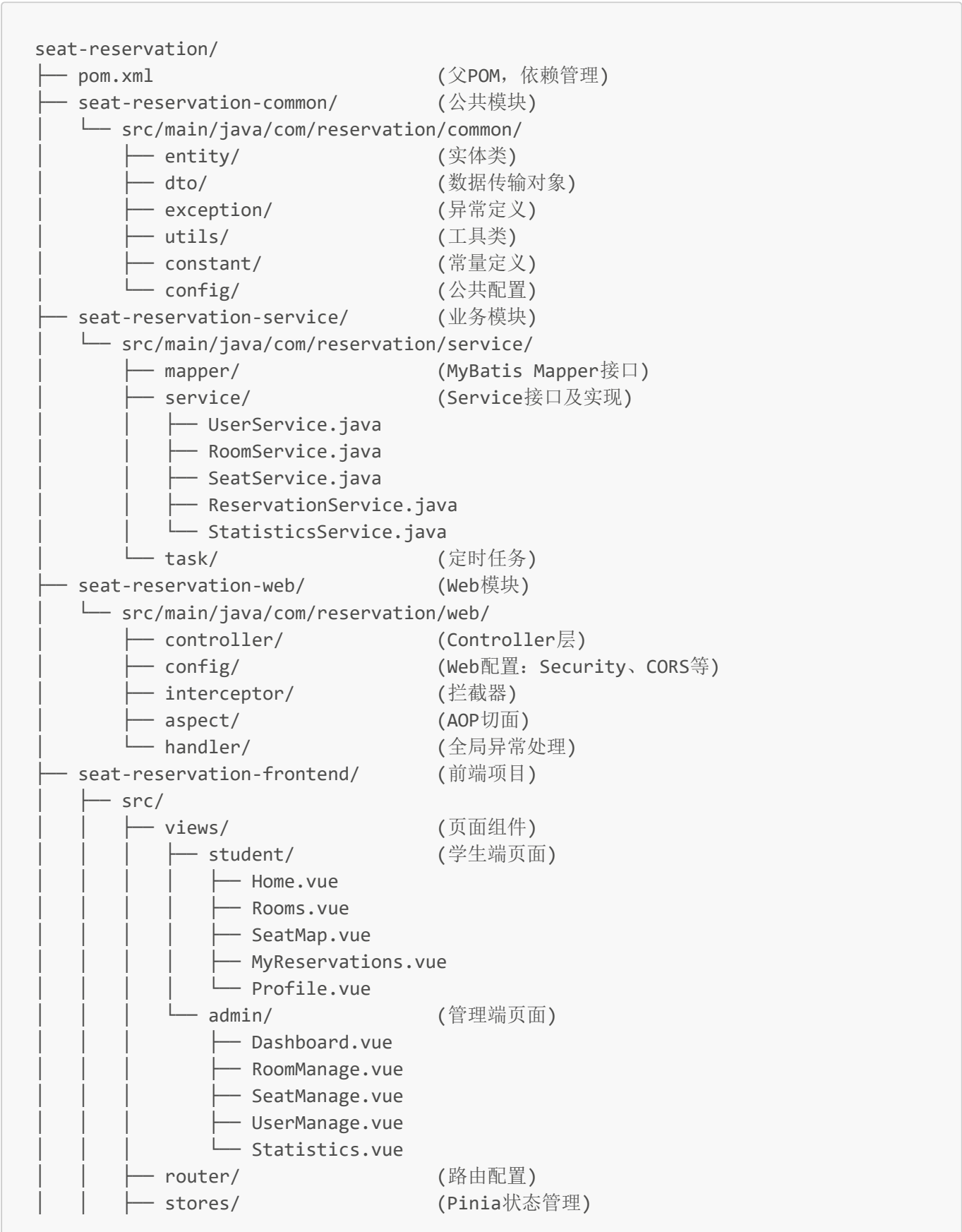
关键监控指标：

- API QPS（每秒请求数）和响应时间P99
- JVM堆内存使用率（告警阈值：>85%）
- 数据库连接池使用率（告警阈值：>80%）

- 活跃用户数、预约量（业务监控）
- 磁盘使用率（告警阈值：>90%）

附录

附录A：项目结构（Maven多模块）





附录B：开发与运行环境要求

环境	版本要求
JDK	17+
Node.js	18+
MySQL	8.0+
Maven	3.9+
Docker	24+ (可选)
操作系统	Windows 10+ / Linux (CentOS 7+, Ubuntu 20.04+)

附录C：参考文献

- 《软件工程》（第10版，Ian Sommerville）
- 《UML用户指南》（Grady Booch 等）
- IEEE Std 830-1998 《软件需求规格说明书推荐实践》
- Spring Boot 官方文档 — <https://spring.io/projects/spring-boot>
- Vue.js 官方文档 — <https://vuejs.org>
- MyBatis-Plus 官方文档 — <https://baomidou.com>
- RESTful API 设计最佳实践 — Microsoft REST API Guidelines

文档编写日期： 2026年6月4日